

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Создание информационной системы для поиска и предоставления услуг
в студенческой среде университета ИТМО.

«HelpMore»

Курсовая работа по дисциплине «Информационные системы»

Этап №4

Научный руководитель:

Тюрин Иван Николаевич

Выполнили:

Покалюхин Илья Игоревич

Лашкул Андрей Владимирович

Группа: Р3310

Санкт-Петербург, 2025 г.

Оглавление

Этап 1	5
1. Согласование предметной области	5
2. Подробное описание предметной области	5
Сущности предметной области	5
Ключевые аспекты предметной области	6
1. Услуги	6
2. Пользователи системы	7
3. Категории услуг	7
4. Заказы и предложения	8
5. Оценки, рейтинги и отзывы	8
6. Поиск и фильтрация	8
7. Коммуникация	8
3. Назначение ИС и решаемые задачи	8
4. Требования к системе	9
Функциональные требования (неавторизованный пользователь)	9
Функциональные требования (авторизованный пользователь)	9
Функциональные требования (модератор)	10
Функциональные требования (администратор)	10
Нефункциональные требования	11
U1: Удобство использования	11
R2: Надежность	11
P3: Производительность	11
S4: Поддерживаемость	11
T5: Тестирование	11
5. Модели основных прецедентов	12
Прецедент U001	12
Прецедент U002	12
Прецедент U003	13
Прецедент U004	14
Прецедент U005	14
Прецедент U006	15
Прецедент U007	15

Прецедент U008	16
Прецедент U009	16
Прецедент U010	17
Предлагаемая архитектура и стек технологий	17
Архитектура	17
Основные модули бэкенда	18
Стек	18
6. Вывод	18
Этап 2	19
1. ER-модель	19
2. Дatalogическая модель	20
3. Скрипты для создания, удаления базы данных	21
Удаление БД	21
Создание БД	21
Создание таблиц	21
4. Заполнение базы тестовыми данными	25
5. pl/pgsql-функции и процедуры	27
6. Индексы	30
app_user (U001/U002): авторизация и регистрация	30
service (U003–U005, U009): публикация и поиск услуг/запросов	30
response (U006): отклики	31
message (U007): сообщения	31
feedback (U008–U009): отзывы/рейтинги	32
report и bug_report (U010): модерация	32
Вывод	33
Этап 3	33
Общая архитектура системы	33
Диаграмма классов User Management Service	35
Диаграмма классов Management Service	38
Диаграмма классов Communication Service	42
Диаграмма классов API Gateway	43
Вывод	43
Анализ выполнения требований	44
Процент выполненных требований	49

Функциональные требования	49
Не функциональные требования	49
Общий анализ	49
Вывод	49

Этап 1

1. Согласование предметной области

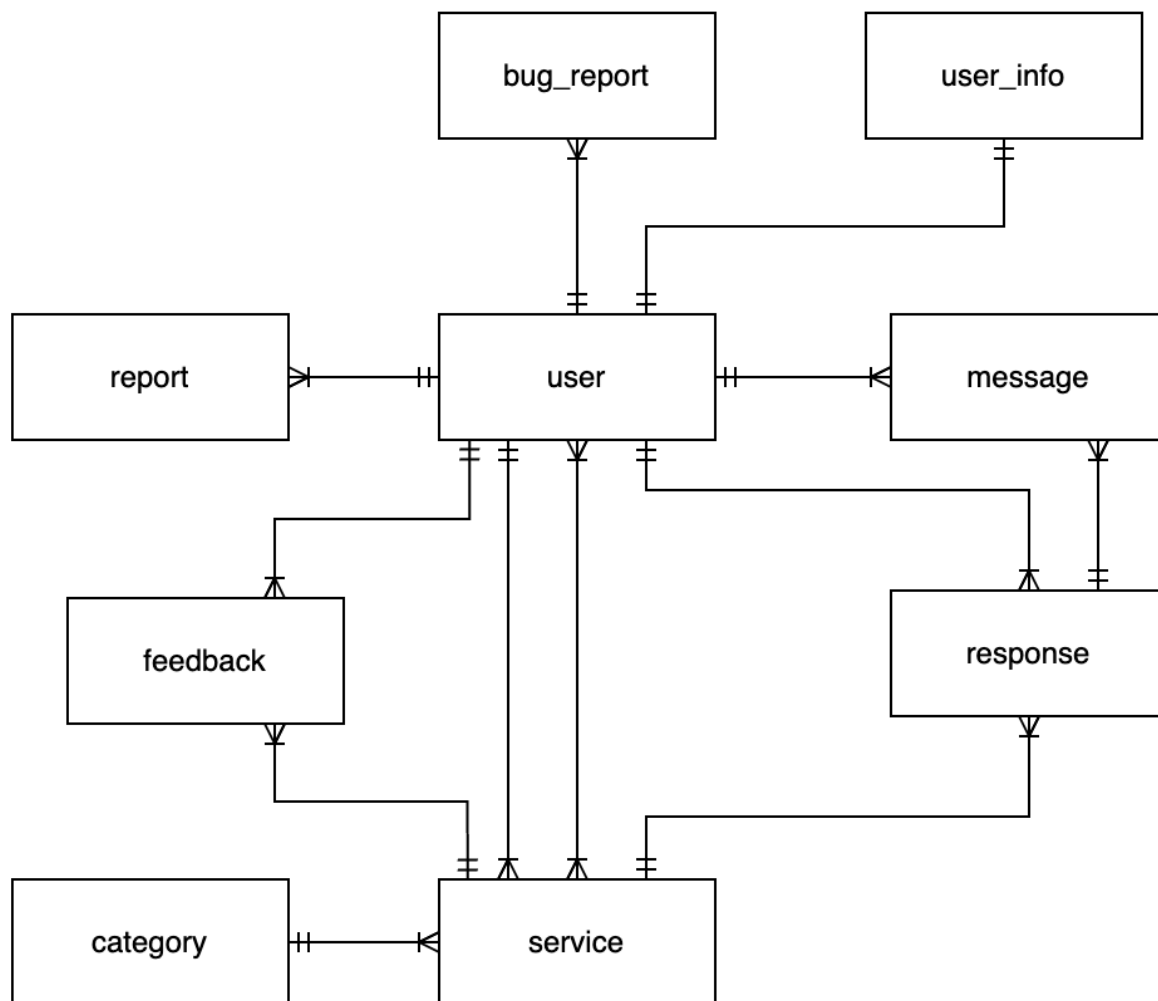
Информационная система для поиска и предоставления услуг в студенческой среде университета ИТМО, «HelpMore».

2. Подробное описание предметной области

HelpMore — это информационная система для поиска и предоставления услуг в студенческой среде университета ИТМО. Она позволяет пользователям размещать запросы на помощь, предлагать собственные услуги, находить исполнителей или консультантов по различным категориям (учебные задания, бытовые дела, карьерные консультации, социальная поддержка и др.), а также оценивать качество оказанных услуг, оставлять отзывы и формировать собственную репутацию внутри сообщества. Система предоставляет инструменты для поиска и фильтрации услуг, организации коммуникации между участниками и безопасного проведения сделок.

Сущности предметной области

- Пользователи
- Избранное
- Услуги
 - Заказы
 - Предложения
- Категории
- Отзывы
- Жалобы
- Сообщения об ошибках (bug-report)
- Диалоги
 - Сообщения



Ключевые аспекты предметной области

1. Услуги

Услуга — это действие или помощь, предоставляемая одним участником студенческого сообщества другому в рамках университетской среды.

Каждая услуга имеет ключевые атрибуты:

- Название услуги
- Категория
 - Учебная
 - Бытовая
 - Карьерная
 - Социальная
- Описание
- Заказчик (лицо, запрашивающее услугу)
- Исполнитель (лицо, предлагающее услугу)

- Условия оказания (время, место, формат)
- Возможная стоимость или форма обмена (бесплатно, за деньги, бартер).

Услуги имеют метаданные, такие как:

- отзывы;
- статус (активна, выполнена, недоступна);
- дата публикации;

Эти данные помогают студентам быстро находить нужную помощь, оценивать качество и надежность исполнителей, а также формировать доверие в сообществе.

2. Пользователи системы

- Зарегистрированные пользователи (лица, относящиеся к ИТМО) — ключевые участники, которые могут выступать как в роли заказчика (размещать запросы на помощь), так и в роли исполнителя (предлагать услуги и откликаться на заказы).
- Модераторы — пользователи с расширенными правами, которые рассматривают жалобы и принимают решения в ответ на жалобы.
- Администраторы — пользователи с полным доступом к системе: управление категориями услуг, управление ролями пользователей.

Профиль каждого пользователя включает:

- личные данные (имя, факультет, курс, контактная информация);
- описание профиля (bio);
- историю заказов и оказанных услуг;
- рейтинг и отзывы.

3. Категории услуг

- Учебные услуги — помощь с домашними и лабораторными работами, объяснение теории, подготовка к экзаменам, проверка и доработка курсовых и дипломных проектов.
 - Учебные дисциплины (Например: Математический анализ, Линейная алгебра)
 - ВКР
- Бытовые услуги:
 - Переезд
 - Доставка
 - Ремонт
 - Настройка техники
 - Покупки
- Карьерные услуги:
 - Консультации по стажировкам

- Подготовка резюме
- Тренировка собеседований.
- Социальные и информационные услуги:
 - Ответы по адаптации в университете
 - Обмен опытом участия в мероприятиях
 - Волонтёрских программах или студенческих клубах.

Категории могут редактироваться и дополняться администраторами в зависимости от потребностей студентов.

4. Заказы и предложения

- **Заказ** — это запрос на оказание услуги, содержащий описание задачи, стоимость и пожелания к исполнителю.
- **Предложение** — это инициатива исполнителя, указывающая доступные услуги и условия их предоставления.

Платформа двусторонняя:

- заказчик ищет исполнителя, публикуя задачу;
- исполнитель ищет заказчика, предлагая свои компетенции.

5. Оценки, рейтинги и отзывы

- Оценка — числовое выражение качества выполнения предоставленных услуг, выставяемое после завершения заказа.
- Рейтинг — среднее арифметическое по всем оценкам выполненных услуг.
- Отзывы — текстовые комментарии, отражающие впечатления заказчика от взаимодействия.

6. Поиск и фильтрация

Поиск и фильтрация по:

- категории (учёба, быт, карьера и т.д.);
- предмету или типу задачи (например, «Математический анализ», «помощь с переездом»);
- рейтингу исполнителя;
- стоимости.

7. Коммуникация

Зарегистрированные пользователи общаются через встроенный чат, обсуждают детали заказа, делятся изображениями.

3. Назначение ИС и решаемые задачи

ИС **HelpMore** предназначена для поиска и предоставления услуг в студенческой среде ИТМО, организации взаимодействия между участниками, обмена опытом и формированием репутации на основе оценок и отзывов. Система создаёт единое пространство для упорядочивания и

оптимизации процессов взаимопомощи, облегчает поиск подходящих услуг и исполнителей и предоставляет функции для удобного и безопасного взаимодействия.

4. Требования к системе

Функциональные требования (неавторизованный пользователь)

- F1.1 Система должна предоставлять неавторизованному пользователю возможность входа по протоколу OAuth 2.0 через Яндекс ID.
- F1.2 Система должна предоставлять неавторизованному пользователю возможность выбора языка в хэдере на всех страницах (включая неавторизованных пользователей).

Функциональные требования (авторизованный пользователь)

- F2.1 Система должна предоставлять авторизованному пользователю возможность просмотра открытых предложений и заказов.
- F2.2 Система должна предоставлять авторизованному пользователю возможность просмотра профилей пользователей (ФИО, bio, контакты, рейтинг и отзывы, количество выполненных заказов).
- F2.3 Система должна предоставлять авторизованному пользователю возможность редактирования личного профиля (bio, контакты).
- F2.4 Система должна предоставлять авторизованному пользователю возможность создания и публикации заказов (название, описание, категория, стоимость/бартер).
- F2.5 Система должна предоставлять авторизованному пользователю возможность создания публикации предложений услуг (название, описание, категория, стоимость).
- F2.6 Система должна предоставлять авторизованному пользователю возможность поиска и фильтрации открытых заказов (категория, стоимость).
- F2.7 Система должна предоставлять авторизованному пользователю возможность поиска и фильтрации открытых предложений (категория, рейтинг, стоимость).
- F2.8 Система должна предоставлять авторизованному пользователю возможность отклика на заказы и связи с заказчиками.
- F2.9 Система должна предоставлять авторизованному пользователю возможность отклика на предложения и связи с поставщиками.
- F2.10 Система должна предоставлять авторизованному пользователю возможность ведения чата с другими пользователями (обсуждение условий, обмен изображениями).
- F2.11 Система должна предоставлять авторизованному пользователю возможность оставления оценок и текстовых отзывов после выполнения услуги.
- F2.12 Система должна предоставлять авторизованному пользователю возможность просмотра рейтинга и отзывов всех пользователей.

- F2.13 Система должна предоставлять авторизованному пользователю возможность просмотра личной истории заказов и оказанных предложений.
- F2.14 Система должна предоставлять авторизованному пользователю возможность отметки интересных заказов/предложений (избранное).
- F2.15 Система должна предоставлять авторизованному пользователю возможность подачи жалоб на некорректное поведение или нарушение правил.
- F2.16 Система должна предоставлять авторизованному пользователю возможность выхода из аккаунта.
- F2.17 Система должна предоставлять авторизованному пользователю возможность наследовать права неавторизованного пользователя.
- F2.18 Система должна предоставлять авторизованному пользователю возможность выбора языка интерфейса в хэдере.

Функциональные требования (модератор)

- F3.1 Система должна предоставлять модератору возможность удаления некорректных заказов/предложений.
- F3.2 Система должна предоставлять модератору возможность рассмотрения жалоб пользователей.
- F3.3 Система должна предоставлять модератору возможность выдачи временной блокировки пользователей при нарушении правил.
- F3.4 Система должна предоставлять модератору возможность наследовать права пользователя.

**Роль модератора включает права пользователя.*

Функциональные требования (администратор)

- F4.1 Система должна предоставлять администратору возможность управления категориями услуг (создание, редактирование, удаление).
- F4.2 Система должна предоставлять администратору возможность управления ролями пользователей (назначение модераторов, снятие прав).
- F4.3 Система должна предоставлять администратору возможность удаления или восстановления учетных записей пользователей.
- F4.4 Система должна предоставлять администратору возможность наследовать права модератора.

**Роль администратора включает права модератора.*

Нефункциональные требования

U1: Удобство использования

- U1.1 Система должна поддерживать доступность через ARIA-атрибуты.
- U1.2 Система должна обеспечивать мультиязычность интерфейса..
- U1.3 Система должна отображать индикатор процесса при загрузке файлов или больших данных.

R2: Надежность

- R2.1 Система должна обеспечивать доступность не менее 95% времени в году.
- R2.2 Система должна обеспечивать единый учёт отзывов, оценок, количества выполненных заказов.
- R2.3 Система должна поддерживать мониторинг (с использованием Grafana).

P3: Производительность

- P3.1 Система должна предоставлять первые результаты поиска ≤ 2 секунды (с использованием Hibernate search).
- P3.2 Система должна обеспечивать пропускную способность: не менее 100 запросов просмотров услуг в секунду в течение минуты, не менее 50 публикаций в секунду в течение минуты.
- P3.3 Система должна загружать любую страницу за 1–2 секунды при 10 Мбит/с.
- P3.4 Система должна реализовывать кэширование данных.

S4: Поддерживаемость

- S4.1 Система должна быть реализована на основе сервисно-ориентированной архитектуры (SOA)
- S4.2 Система должна разрабатываться по принципу API-first, и предоставлять публичные и внутренние API.
- S4.3 Система должна реализовывать RESTful API (JSON), должны поддерживаться стандартные HTTP-методы (GET, POST, PUT, DELETE).
- S4.4 Система должна обладать документацией к внутреннему и внешнему API (с использованием OpenAPI).
- S4.5 Система должна легко разворачиваться за счёт контейнеризации (с использованием Docker).
- S4.6 Система должна осуществлять мониторинг и алертинг (с использованием Grafana).
- S4.7 Система должна поддерживать интернационализацию пользовательского интерфейса с возможностью выбора одного из трёх языков: русского, английского и китайского.

T5: Тестирование

- T5.1 Система должна иметь модульные тесты для ключевой бизнес логики.

- T5.2 Система должна иметь настроенный CI/CD через github actions при коммите в ветку разработки.
- T5.3 Система должна проверять доступность ключевых сервисов для корректной работы приложения (настройка через docker-compose).

5. Модели основных прецедентов

Прецедент U001

Прецедент	U001
ID	1
Краткое описание	Пользователь авторизуется в системе через Яндекс ID
Главный актер	Пользователь
Второстепенные акторы	Система, Сервис авторизации Яндекс ID
Предусловия	Пользователь имеет учётную запись Яндекс ID и не авторизован в системе HelpMore
Основной поток	1. Пользователь нажимает кнопку «Войти через Яндекс ID» на главной странице HelpMore. 2. Система перенаправляет пользователя на страницу авторизации Яндекс ID. 3. Пользователь подтверждает вход, используя свой Яндекс-аккаунт. 4. Сервис Яндекс ID проверяет данные и возвращает токен аутентификации в систему HelpMore. 5. Система принимает токен, проверяет его подлинность и аутентифицирует пользователя. 5. При успешной проверке система авторизирует пользователя. 6. Система перенаправляет пользователя на главную страницу.
Альтернативный поток	A1. Ошибка авторизации в Яндекс ID. • Система отображает сообщение об ошибке. • Пользователь может повторить попытку авторизации. A2. Пользователь отменяет вход через Яндекс ID. – Система возвращает пользователя на страницу авторизации в сервис Helpmore.

Прецедент U002

Прецедент	U002
ID	2

Краткое описание	Пользователь регистрируется в системе через Яндекс ID
Главный актер	Новый пользователь
Второстепенные акторы	Система, Сервис авторизации Яндекс ID
Предусловия	Пользователь не зарегистрирован в системе Helpmore.
Основной поток	1. Пользователь нажимает кнопку «Войти через Яндекс ID» на главной странице HelpMore. 2. Система перенаправляет пользователя на страницу авторизации Яндекс ID. 3. Пользователь подтверждает доступ к своим данным (имя, email и т.д.). 4. Сервис Яндекс ID передаёт системе HelpMore информацию о пользователе и токен доступа. 5. Система проверяет, существует ли пользователь с таким email в базе данных. 5. При успешной проверке система создаёт новую учётную запись и связывает её с Яндекс ID. 6. Система уведомляет пользователя об успешной регистрации и выполняет автоматическую авторизацию.
Альтернативный поток	A1. Пользователь уже зарегистрирован с данным Яндекс ID или email. • Система авторизует пользователя через Яндекс ID и перенаправляет на главную страницу. A2. Пользователь отменяет регистрацию через Яндекс ID. – Система возвращает пользователя на страницу авторизации в сервис Helpmore.

Прецедент U003

Прецедент	U003
ID	3
Краткое описание	Пользователь создает запрос на помощь
Главный актер	Пользователь (Заказчик)
Второстепенные акторы	Система
Предусловия	Пользователь авторизован и не заблокирован в системе.
Основной поток	1. Пользователь переходит в раздел «Создать запрос». 2. Система отображает форму с полями: категория (учебная/бытовая и т.д.), заголовок, подробное описание, бюджет (если есть).

	3. Пользователь заполняет форму и публикует запрос. 4. Система проверяет данные на корректность и размещает запрос в общей ленте. 5. Пользователь попадает на страницу с предлагаемыми услугами
Альтернативный поток	A1. Данные не прошли проверку. Система отображает ошибку и предлагает исправить поля.
Постусловия	Новый заказ опубликован и виден другим пользователям.

Прецедент U004

Прецедент	U004
ID	4
Краткое описание	Пользователь предлагает свою услугу
Главный актер	Пользователь (Исполнитель)
Второстепенные акторы	Система
Предусловия	Пользователь авторизован и не заблокирован в системе.
Основной поток	1. Пользователь переходит в раздел «Предложить услугу». 2. Система отображает форму с полями: категория, название услуги, описание, цена (или бесплатно), доступность. 3. Пользователь заполняет форму и публикует предложение. 4. Система проверяет данные и размещает предложение в каталоге услуг и в профиле пользователя. 5. Пользователь попадает на страницу с заказами
Альтернативный поток	A1. Данные не прошли проверку. Система отображает ошибку и предлагает исправить поля.
Постусловия	Новая услуга опубликована и видна другим пользователям.

Прецедент U005

Прецедент	U005
ID	5
Краткое описание	Пользователь ищет услугу или запрос
Главный актер	Пользователь
Второстепенные акторы	Система
Предусловия	Пользователь авторизован в системе.
Основной поток	1. Пользователь открывает главную страницу или раздел «Поиск».

	2. Пользователь применяет фильтры (категория, ключевые слова, цена, рейтинг исполнителя). 3. Система отображает список соответствующих запросов или услуг. 4. Пользователь просматривает результаты поиска.
Альтернативный поток	A1. Ничего не найдено. Система предлагает изменить параметры поиска или создать заявку на выполнение или оказание услуги.
Постусловия	Найдены подходящие предложения или запросы.

Прецедент U006

Прецедент	U006
ID	6
Краткое описание	Пользователь откликается на запрос или предлагаемую услугу
Главный актер	Пользователь (Исполнитель или Клиент)
Второстепенные акторы	Система, Второй пользователь
Предусловия	Пользователь авторизован и нашел интересующий его запрос/услугу.
Основной поток	1. Пользователь нажимает кнопку «Предложить помощь» (на запросе) или «Написать» (на предложении услуги). 2. Система предоставляет возможность написать сопроводительное сообщение. 3. Пользователь отправляет сообщение. 4. Система уведомляет автора запроса/услуги о новом отклике.
Альтернативный поток	A1. Пользователь пытается откликнуться на собственный запрос. Система блокирует действие и показывает ошибку.
Постусловия	Создан новый отклик. Начато общение между пользователями.

Прецедент U007

Прецедент	U007
ID	7
Краткое описание	Пользователи общаются во внутреннем чате
Главный актер	Пользователь
Второстепенные акторы	Система, Второй пользователь
Предусловия	Пользователи авторизованы. Между ними инициирован контакт (через отклик или напрямую).
Основной поток	1. Пользователь открывает раздел «Сообщения» и выбирает чат. 2. Пользователь отправляет текстовое сообщение или файл.

	3. Система доставляет сообщение собеседнику и сохраняет его в истории переписки. 4. Собеседник получает уведомление о новом сообщении.
Альтернативный поток	A1. Пользователь пытается откликнуться на собственный запрос. Система блокирует действие и показывает ошибку.
Постусловия	Создан новый отклик. Начато общение между пользователями.

Прецедент U008

Прецедент	U008
ID	8
Краткое описание	Пользователи подтверждают завершение услуги и заказчик оставляет отзыв
Главный актер	Пользователь (Клиент и Исполнитель)
Второстепенные акторы	Система
Предусловия	Услуга оказана.
Основной поток	1. Система отправляет обоим пользователям запрос на подтверждение завершения сделки. 2. Пользователи подтверждают завершение. 3. Система открывает возможность оставить отзыв и оценку (от 1 до 5 звезд) друг другу. 4. Пользователи оставляют отзывы. 5. Система обновляет рейтинги пользователей на основе полученных оценок и публикует отзывы.
Альтернативный поток	A1. Пользователи не подтверждают завершение. Система напоминает о необходимости оставить отзыв через определенное время, а затем закрывает сделку автоматически.
Постусловия	Сделка завершена. Репутация обоих пользователей обновлена.

Прецедент U009

Прецедент	U009
ID	9
Краткое описание	Пользователь управляет своим профилем и репутацией
Главный актер	Пользователь
Второстепенные акторы	Система
Предусловия	Пользователь авторизован в системе.
Основной поток	1. Пользователь переходит в свой профиль.

	2. Пользователь редактирует информацию о себе, навыки, контактные данные. 3. Пользователь просматривает свою историю сделок, полученные отзывы и текущий рейтинг. 4. Система сохраняет изменения и пересчитывает рейтинг при получении новых отзывов.
Альтернативный поток	-
Постусловия	Информация в профиле актуализирована.

Прецедент U010

Прецедент	U010
ID	10
Краткое описание	Администратор модерирует пользовательский контент
Главный актер	Администратор/Модератор
Второстепенные акторы	Система
Предусловия	Администратор авторизован в системе с повышенными правами.
Основной поток	1. Администратор открывает панель модерации. 2. Система отображает очередь жалоб от пользователей или автоматически помеченных публикаций. 3. Администратор проверяет контент (запрос, услугу, отзыв) на соответствие правилам платформы. 4. Администратор удаляет нарушающий правила контент и/или применяет санкции к пользователю (предупреждение, блокировка). 5. Система уведомляет пользователя о действиях модератора.
Альтернативный поток	A1. Контент не нарушает правила. Администратор снимает пометку «на проверке».
Постусловия	Контент на платформе соответствует правилам сообщества.

Предлагаемая архитектура и стек технологий

Архитектура

Для построения ИС «HelpMore» используется сервис-ориентированная архитектура (SOA) из нескольких взаимосвязанных сервисов, общающихся с помощью REST API или gRPC. Такой подход повышает гибкость и масштабируемость и позволяет вести параллельную разработку. Клиент-серверная архитектура: frontend и backend разделены и взаимодействуют через API (REST).

- Фронтенд (CSR) с адаптивным дизайном, интернационализацией (RU/EN/CN).

- Бэкенд по принципам MVC, стратегия разработки bottom-up.

Основные модули бэкенда

- Api gateway:
 - Маршрутизация запросов
 - Возврат ответов
 - Валидация JWT
- User Management Service:
 - Аутентификация/Авторизация
 - Роли пользователей
 - Профили пользователей
 - Права пользователей
 - CRUD-операции с пользователями
- Communication service:
 - Общение через чат
- Management service:
 - CRUD-операции с услугами
 - Фильтр услуг
 - Управление статусами услуг
 - Управление оценками услуг

Стек

Фронтенд:

- ReactJS
- Jotai
- Ant Design

Бэкенд:

- Java (Spring Boot)
- Go (только коммуникационный сервис)
- Hibernate
- PostgreSQL
- Redis
- Gradle

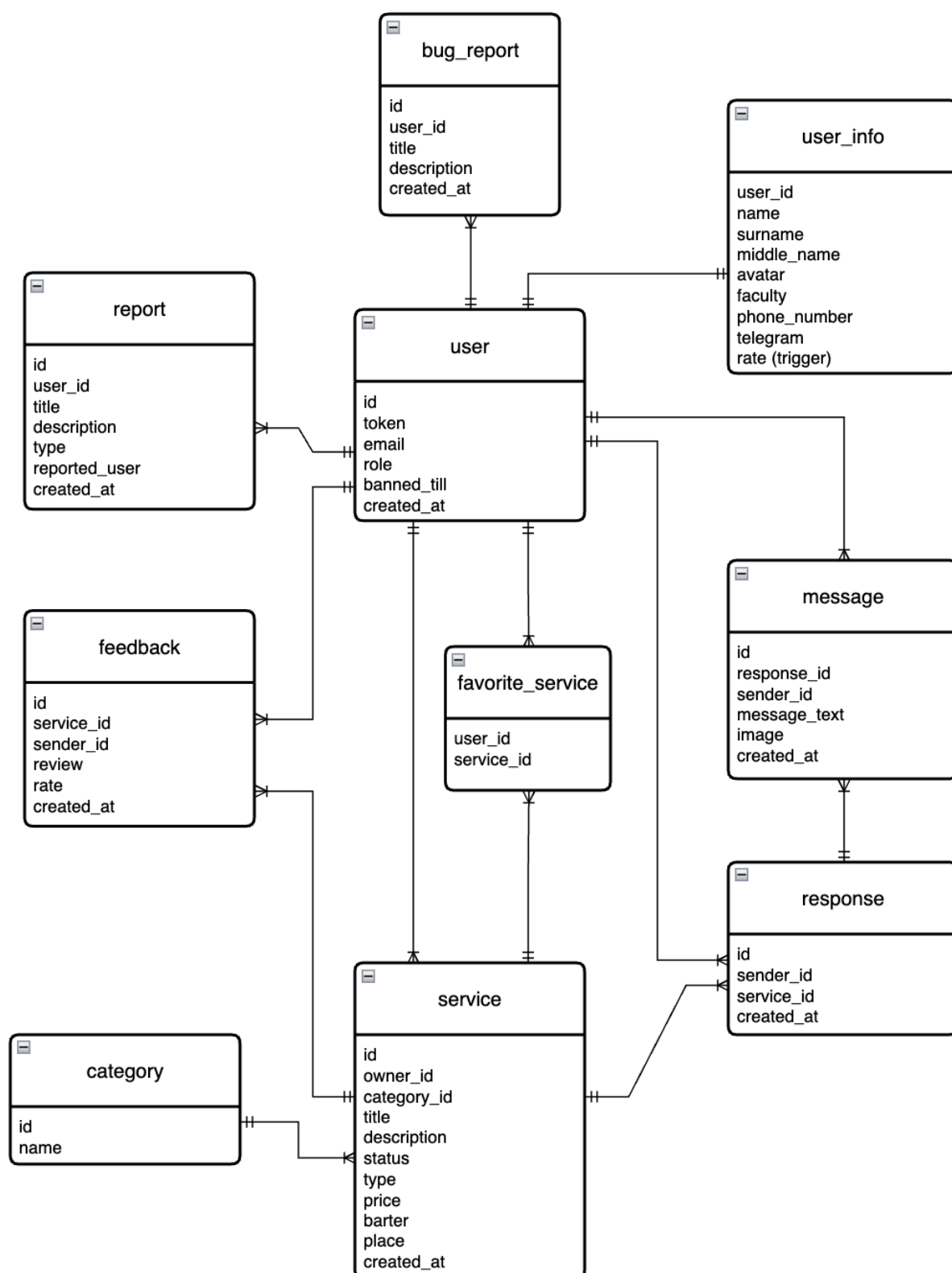
6. Вывод

На первом этапе курсовой работы определена и изучена предметная область, сформулированы функциональные и нефункциональные требования, представлены модели ключевых

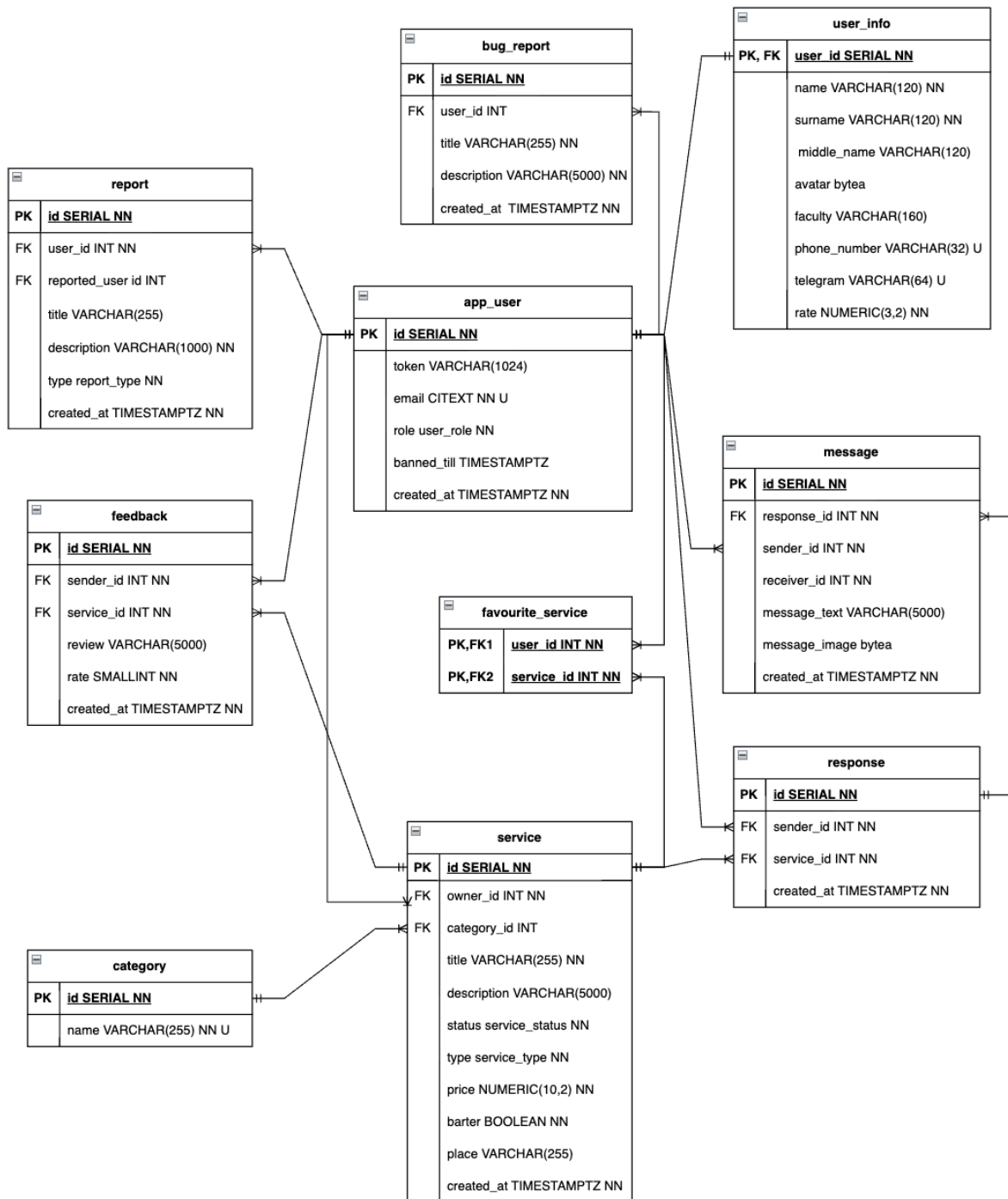
прецедентов, а также предложена архитектура системы и технологический стек. Этап завершён, база для дальнейшего детального анализа и проектирования подготовлена.

Этап 2

1. ER-модель



2. Даталогическая модель



3. Скрипты для создания, удаления базы данных

Удаление БД

DO \$\$

BEGIN

PERFORM pg_terminate_backend(pid)

FROM pg_stat_activity

WHERE datname = 'uniprofi';

EXCEPTION WHEN others THEN

NULL;

END\$\$;

DROP DATABASE IF EXISTS helpmore;

Создание БД

CREATE DATABASE helpmore

WITH ENCODING 'UTF8'

TEMPLATE template0

LC_COLLATE 'C'

LC_CTYPE 'C';

Создание таблиц

CREATE EXTENSION IF NOT EXISTS citext;

BEGIN;

-- ===== Справочные ENUM'ы =====

DO \$\$

BEGIN

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'user_role') THEN

CREATE TYPE user_role AS ENUM ('user','moderator','admin');

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'service_status') THEN

CREATE TYPE service_status AS ENUM ('active',,'archived');

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'service_type') THEN

CREATE TYPE service_type AS ENUM ('offer','order');

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'report_type') THEN

CREATE TYPE report_type AS ENUM ('spam','fraud','insult','illegal','other');

END IF;

END\$\$;

-- ===== Пользователь =====

```
CREATE TABLE app_user (  
    id          SERIAL PRIMARY KEY,  
    token       VARCHAR(1024) NOT NULL UNIQUE,  
    email       CITEXT NOT NULL UNIQUE,  
    role        user_role NOT NULL DEFAULT 'user',  
    banned_till TIMESTAMPTZ,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE user_info (  
    id          SERIAL PRIMARY KEY  
    user_id     INT REFERENCES app_user(id) ON DELETE CASCADE,  
    name        VARCHAR(120) NOT NULL,  
    surname     VARCHAR(120) NOT NULL,  
    middle_name VARCHAR(120),  
    avatar      BYTEA,  
    faculty     VARCHAR(160),  
    phone_number VARCHAR(32),  
    telegram    VARCHAR(64) UNIQUE,  
    rate        NUMERIC(3,2) NOT NULL DEFAULT 0.00,  
    CONSTRAINT chk_phone_fmt CHECK (phone_number IS NULL OR phone_number ~  
'^[0-9+()\-\s]{5,32}$'),  
    CONSTRAINT chk_rate_bounds CHECK (rate >= 0 AND rate <= 5)  
);
```

-- ===== Категории =====

```
CREATE TABLE category (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(255) NOT NULL UNIQUE  
);
```

-- ===== Услуги =====

```
CREATE TABLE service (  
    id          SERIAL PRIMARY KEY,  
    owner_id    INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,  
    category_id INT NOT NULL REFERENCES category(id),
```

```

title          VARCHAR(255) NOT NULL,
description    VARCHAR(5000) NOT NULL,
status        service_status NOT NULL DEFAULT 'active',
type          service_type  NOT NULL,
price         NUMERIC(10,2)  NOT NULL DEFAULT 0.00,
barter        BOOLEAN        NOT NULL DEFAULT FALSE,
place         VARCHAR(255),
created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT chk_price_nonneg CHECK (price >= 0),
CONSTRAINT unq_service_title_per_owner UNIQUE (owner_id, title)
);

```

-- ===== Избранное =====

```

CREATE TABLE favourite_service (
    user_id    INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    service_id INT NOT NULL REFERENCES service(id)  ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, service_id)
);

```

-- ===== Отклики =====

```

CREATE TABLE response (
    id          SERIAL PRIMARY KEY,
    sender_id   INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE, -- кто
откликнулся
    service_id  INT NOT NULL REFERENCES service(id)  ON DELETE CASCADE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT unq_response UNIQUE (sender_id, service_id)
);

```

-- ===== Сообщения по отклику (чат 1-на-1) =====

```

CREATE TABLE message (
    id          SERIAL PRIMARY KEY,
    response_id INT NOT NULL REFERENCES response(id) ON DELETE CASCADE,
    sender_id   INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    receiver_id INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    message_text VARCHAR(5000),
    message_image BYTEA,

```

```

created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT message_not_empty CHECK (
    message_text IS NOT NULL OR message_image IS NOT NULL
)
CONSTRAINT sender_not_receiver CHECK (
    sender_id <> receiver_id
),
);

-- ===== Отзывы =====
CREATE TABLE feedback (
    id          SERIAL PRIMARY KEY,
    sender_id   INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    service_id  INT NOT NULL REFERENCES service(id) ON DELETE CASCADE,
    review      VARCHAR(5000),
    rate        SMALLINT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_rate_range CHECK (rate BETWEEN 1 AND 5),
    CONSTRAINT unq_feedback_one_per_user UNIQUE (sender_id, service_id)
);

-- ===== Репорты и баг-репорты =====
CREATE TABLE report (
    id          SERIAL PRIMARY KEY,
    user_id     INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    reported_user_id INT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    title       VARCHAR(255) NOT NULL,
    description  VARCHAR(1000) NOT NULL,
    type        report_type NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_report_not_self CHECK (user_id <> reported_user_id)
);

CREATE TABLE bug_report (
    id          SERIAL PRIMARY KEY,
    user_id     BIGINT NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    title       VARCHAR(255) NOT NULL,
    description  VARCHAR(5000) NOT NULL,

```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
COMMIT;
```

4. Заполнение базы тестовыми данными

```
BEGIN;
```

```
-- ===== Пользователи =====
```

```
INSERT INTO app_user (token, email, role)
```

```
VALUES
```

```
    ('token_user1', 'user1@example.com', 'user'),
    ('token_user2', 'user2@example.com', 'user'),
    ('token_user3', 'user3@example.com', 'user'),
    ('token_admin', 'admin@example.com', 'admin');
```

```
-- ===== Информация о пользователях =====
```

```
INSERT INTO user_info (user_id, name, surname, faculty, telegram)
```

```
VALUES
```

```
    (1, 'Иван', 'Иванов', 'ИТМО', '@ivan_itmo'),
    (2, 'Пётр', 'Петров', 'ИТМО', '@petr_itmo'),
    (3, 'Сергей', 'Сергеев', 'ИТМО', '@serg_itmo'),
    (4, 'Админ', 'Системный', 'ИТМО', '@admin_itmo');
```

```
-- ===== Категории =====
```

```
INSERT INTO category (name)
```

```
VALUES
```

```
    ('Программирование'),
    ('Репетиторство'),
    ('Дизайн'),
    ('Ремонт'),
    ('Моск-собеседование'),
    ('Карьера'),
    ('Другое');
```

```
-- ===== Услуги =====
```

```
-- Иван (user1) предлагает услугу "Написание курсовых"
```

```
INSERT INTO service (owner_id, category_id, title, description, type, price)
```

```
VALUES
```

```
    (1, 1, 'Помощь с курсовыми по Java', 'Вместе разберемся с различными технологиями:
```

```

Spring data, Spring Boot, SQL и другие.', 'offer', 1500.00),
(1, 2, 'Репетитор по информатике', 'Подготовка к ЕГЭ, ОГЭ, олимпиадам.', 'offer',
1000.00),
(2, 3, 'Логотипы на заказ', 'Делаю простые и стильные логотипы.', 'offer',
1200.00),
(2, 1, 'Помощь с курсовой по Python', 'Нужен студент, который поможет разобраться с
pandas и matplotlib.'),
(3, 4, 'Ремонт ноутбука', 'Перестал включаться ноутбук, требуется диагностика и
ремонт.', 'order', 1000.00),
(3, 1, 'Настройка Docker проекта', 'Есть проект на Spring + React, нужно помочь с
docker-compose.', 'order', 1337.00);

```

```
-- ===== Отклики =====
```

```
-- Пётр откликается на услугу Ивана
```

```
INSERT INTO response (sender_id, service_id)
```

```
VALUES
```

```
(2, 1), -- Пётр откликнулся на "Помощь с курсовыми"
```

```
(3, 1); -- Сергей тоже откликнулся на ту же услугу
```

```
-- ===== Сообщения по отклику =====
```

```
INSERT INTO message (response_id, sender_id, receiver_id, message_text)
```

```
VALUES
```

```
(1, 2, 1, 'Здравствуйте! Хотел бы обсудить детали.'),
```

```
(1, 1, 2, 'Конечно, задавайте вопросы.'),
```

```
(2, 3, 1, 'Здравствуйте, вы делаете проекты по базам данных?');
```

```
-- ===== Отзывы =====
```

```
-- Пётр оставил отзыв Ивану (по первой услуге, 5 звезд)
```

```
INSERT INTO feedback (sender_id, service_id, review, rate)
```

```
VALUES
```

```
(2, 1, 'Отличная работа, всё вовремя и качественно!', 5);
```

```
-- Сергей оставил отзыв Ивану (по первой услуге, 4 звезды)
```

```
INSERT INTO feedback (sender_id, service_id, review, rate)
```

```
VALUES
```

```
(3, 1, 'Хорошо, но были задержки.', 4);
```

```

-- теперь rate у Ивана должен обновиться с 5.00 → (5+4)/2 = 4.50

-- ===== Избранное =====
INSERT INTO favourite_service (user_id, service_id)
VALUES
    (2, 1),
    (3, 1),
    (3, 3);

-- ===== Репорты =====
INSERT INTO report (user_id, reported_user_id, title, description, type)
VALUES
    (2, 3, 'Спам', 'Пользователь пишет в личные сообщения без причины', 'spam'),
    (3, 2, 'Оскорбление', 'В отзыве использованы грубые слова', 'insult');

-- ===== Баг-репорт =====
INSERT INTO bug_report (user_id, title, description)
VALUES
    (1, 'Ошибка загрузки аватара', 'Не загружается файл размером более 2 МБ');

```

```
COMMIT;
```

5. pl/pgsql-функции и процедуры

```

BEGIN;

-- запрет действий забаненных пользователей — вспомогательная функция
CREATE OR REPLACE FUNCTION assert_not_banned(p_user_id INT)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE v_until TIMESTAMPTZ;
BEGIN
    SELECT banned_till INTO v_until FROM app_user WHERE id = p_user_id;
    IF v_until IS NOT NULL AND v_until > now() THEN
        RAISE EXCEPTION 'user % is banned until %', p_user_id, v_until
            USING ERRCODE = 'check_violation';
    END IF;
END$$;

-- триггер: запрет публикации услуги для забаненных
CREATE OR REPLACE FUNCTION trg_service_assert_not_banned()

```

```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    PERFORM assert_not_banned(NEW.owner_id);
    RETURN NEW;
END$$;

CREATE TRIGGER t_service_not_banned
BEFORE INSERT OR UPDATE ON service
FOR EACH ROW EXECUTE FUNCTION trg_service_assert_not_banned();

-- запрет отклика на свой сервис
CREATE OR REPLACE FUNCTION trg_response_no_self()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_owner INT;
BEGIN
    SELECT owner_id INTO v_owner FROM service WHERE id = NEW.service_id;
    IF v_owner = NEW.sender_id THEN
        RAISE EXCEPTION 'owner cannot respond to own service' USING
ERRCODE='check_violation';
    END IF;
    PERFORM assert_not_banned(NEW.sender_id);
    RETURN NEW;
END$$;

CREATE TRIGGER t_response_no_self
BEFORE INSERT ON response
FOR EACH ROW EXECUTE FUNCTION trg_response_no_self();

-- запрет отзыва на свой сервис
CREATE OR REPLACE FUNCTION trg_feedback_no_self()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_owner INT;
BEGIN
    SELECT owner_id INTO v_owner FROM service WHERE id = NEW.service_id;
    IF v_owner = NEW.sender_id THEN
        RAISE EXCEPTION 'owner cannot leave feedback to own service' USING
ERRCODE='check_violation';
    END IF;

```



```

    PERFORM assert_not_banned(NEW.sender_id);
    RETURN NEW;
END$$;

CREATE TRIGGER t_feedback_no_self
BEFORE INSERT ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_no_self();

-- поддержка агрегированного рейтинга исполнителя (owner) в user_info.rate
CREATE OR REPLACE FUNCTION recalc_owner_rate(p_owner_id INT)
RETURNS VOID LANGUAGE plpgsql AS $$
BEGIN
    UPDATE user_info ui
    SET rate = COALESCE((
        SELECT ROUND(AVG(f.rate)::numeric, 2)
        FROM feedback f
        JOIN service s ON s.id = f.service_id
        WHERE s.owner_id = p_owner_id
    ), 0)
    WHERE ui.user_id = p_owner_id;
END$$;

CREATE OR REPLACE FUNCTION trg_feedback_recalc_rate()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_owner INT;
BEGIN
    SELECT owner_id INTO v_owner FROM service WHERE id =
COALESCE(NEW.service_id, OLD.service_id);
    PERFORM recalc_owner_rate(v_owner);
    RETURN COALESCE(NEW, OLD);
END$$;

CREATE TRIGGER t_feedback_recalc_ins
AFTER INSERT ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_recalc_rate();

CREATE TRIGGER t_feedback_recalc_upd

```

```
AFTER UPDATE ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_recalc_rate();
```

```
CREATE TRIGGER t_feedback_recalc_del
AFTER DELETE ON feedback
FOR EACH ROW EXECUTE FUNCTION trg_feedback_recalc_rate();
```

6. Индексы

app_user (U001/U002): авторизация и регистрация

Быстрые проверки бана и поиск по токenu.

```
CREATE INDEX IF NOT EXISTS app_user_banned_active_idx
ON app_user (banned_till)
WHERE banned_till IS NOT NULL;
```

```
CREATE INDEX IF NOT EXISTS idx_app_user_token
ON app_user (token);
```

Индексы необходимы для выполнения требований:

- F1.1, F2.16 — вход/выход через OAuth (Google / Яндекс) и работа с сессиями.
- F2.17, F3.3, F4.3 — проверка, не заблокирован ли пользователь (при логине), а также блокировка и разблокировка пользователей.

Оба индекса btree, потому что он наилучшим подходит для операций =, <, >

service (U003–U005, U009): публикация и поиск услуг/запросов

Для поиска услуг по фильтрам

```
CREATE INDEX IF NOT EXISTS service_pub_cat_type_created_idx
ON service (category_id, type, created_at DESC)
WHERE status = 'active';
```

Для поиска всех активных услуг

```
CREATE INDEX IF NOT EXISTS service_pub_created_idx
ON service (created_at DESC)
WHERE status = 'active';
```

Для просмотра своих услуг

```
CREATE INDEX IF NOT EXISTS service_owner_created_idx
ON service (owner_id, created_at DESC);
```

Для поиска в ценовом диапазоне

```
CREATE INDEX IF NOT EXISTS idx_service_price
ON service (price);
```

Для поиска по названию услуги (title)

```
CREATE INDEX idx_service_title_trgm ON service USING gin (title gin_trgm_ops);
```

Индексы необходимы для выполнения требований:

- F2.1 — просмотр открытых предложений и заказов (лента активных услуг).
- F2.4, F2.5 — публикация заказов и предложений (после вставки нужно быстро видеть их в списке)
- F2.6, F2.7 — поиск и фильтрация по категории и стоимости, и текстовый поиск по названию.
- F2.13 — личная история заказов/предложений (фильтр по owner_id)
- F2.14 — избранное.
- F3.1 — модератору проще находить и удалять проблемные заказы/предложения.

service_pub_cat_type_created_idx, service_pub_created_idx, service_owner_created_idx,

idx_service_price — btree: так как используют точное сравнение

service_pub_* - частичные: так как почти всегда работа ведется с активными услугами

idx_service_title_trgm — GIN + pg_trgm: для поиска по подстроке/опечаткам

response (U006): отклики

Отклики по услуге, с порядком по времени

```
CREATE INDEX IF NOT EXISTS idx_response_service_created_at
ON response (service_id, created_at DESC);
```

Отклики, отправленные пользователем

```
CREATE INDEX IF NOT EXISTS idx_response_sender_created_at
ON response (sender_id, created_at DESC);
```

Индексы необходимы для выполнения требований:

- F2.8, F2.9 — отклик на заказы и предложения:
 - быстрый показ списка откликов на конкретный заказ/услугу.
- F2.13 — история своих откликов.
- F2.10 — старт чата строится от отклика.

Оба индекса btree, потому что он наилучшим подходит для операций =, <, >

message (U007): сообщения

Сообщения по конкретному отклику

```
CREATE INDEX IF NOT EXISTS idx_message_response_id
ON message (response_id);
```

Входящие/исходящие по пользователю с датой

```
CREATE INDEX IF NOT EXISTS idx_message_receiver_created_at
ON message (receiver_id, created_at);
```

```
CREATE INDEX IF NOT EXISTS idx_message_sender_created_at
ON message (sender_id, created_at);
```

```
CREATE INDEX IF NOT EXISTS message_pair_created_idx
ON message (
    LEAST(sender_id, receiver_id),
    GREATEST(sender_id, receiver_id),
    created_at DESC
);
```

Индексы необходимы для выполнения требований:

- F2.10 — чат между пользователями:
 - загрузка истории диалога
 - список входящих/исходящих сообщений.
- F2.8, F2.9 — связи по откликам (чат по конкретному отклику).

Все индексы btree, потому что они наилучшим подходит для операций =, <, >

feedback (U008–U009): отзывы/рейтинги

Отзывы по услуге, свежие сверху

```
CREATE INDEX IF NOT EXISTS idx_feedback_service_created_at
ON feedback (service_id, created_at DESC);
```

Отзывы, оставленные пользователем

```
CREATE INDEX IF NOT EXISTS idx_feedback_sender_created_at
ON feedback (sender_id, created_at DESC);
```

Индексы необходимы для выполнения требований:

- F2.11 — оставление оценок и отзывов.
- F2.2, F2.12 — просмотр рейтинга и отзывов в профиле/на услуге.
- F2.13 — история оставленных отзывов.

Все индексы btree, потому что они наилучшим подходит для операций =, <, >

report и bug_report (U010): модерация

```
CREATE INDEX IF NOT EXISTS report_reported_created_idx
ON report (reported_user_id, created_at DESC);
```

```
CREATE INDEX IF NOT EXISTS report_type_idx
ON report (type);
```

```
CREATE INDEX IF NOT EXISTS bug_report_created_idx
ON bug_report (created_at DESC);
```

```
CREATE INDEX IF NOT EXISTS bug_report_user_idx
ON bug_report (user_id);
```

- F2.15 — подача жалоб пользователями:

- модератор быстро видит жалобы на конкретного пользователя.
- F3.1 — удаление некорректных заказов/предложений.
- F3.2 — рассмотрение жалоб пользователями (очередь жалоб по дате и по типам).
- F3.3 — блокировка нарушителей (быстрая выборка всех жалоб на пользователя).

Все индексы btree, потому что они наилучшим подходит для операций =, <, >

Вывод

В результате выполнения второго этапа курсовой работы по дисциплине "Информационные системы", была разработана структура реляционной базы данных на базе заранее сформулированных ER- и даталогической моделей будущей информационной системы. Полученная структура БД соответствует заданным требованиям и обеспечивает нужный уровень целостности данных. В дальнейшем при необходимости система может быть переработана с учётом новых требований.

Этап 3

Общая архитектура системы

В 1-м этапе была предложена сервисно-ориентированная архитектура (SOA) с разделением фронтенда и бэкенда и модулями бэкенда: API gateway, User Management Service, Communication service, Management service, работающими через REST.

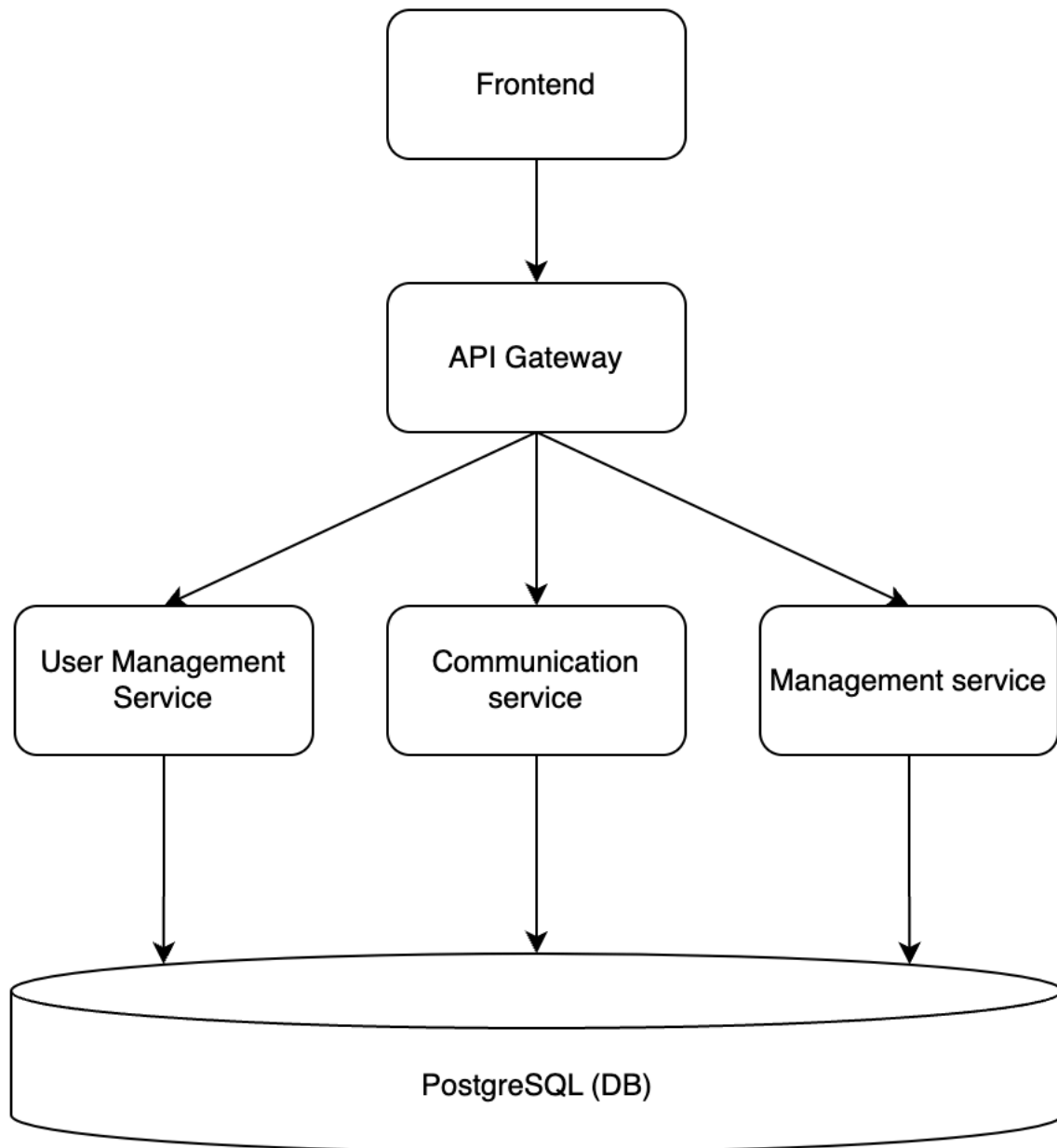


Диаграмма классов User Management Service

Общая диаграмма классов



Диаграмма классов configuration

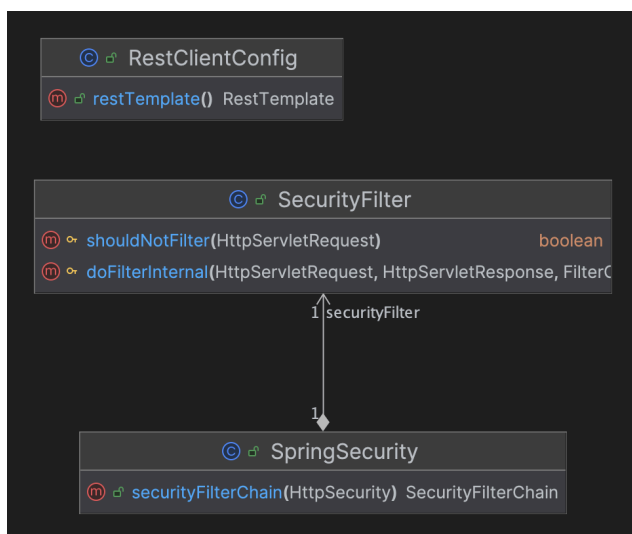


Диаграмма классов controllers

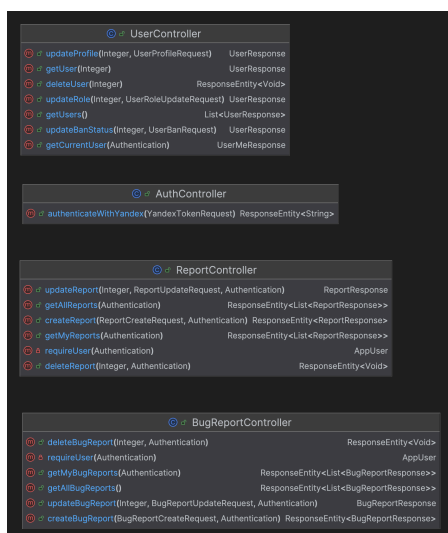


Диаграмма классов models

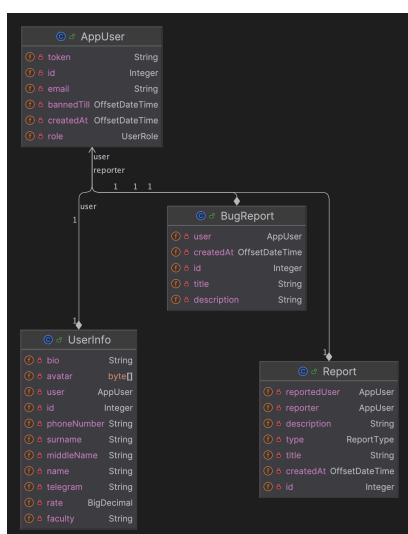


Диаграмма интерфейсов repositories

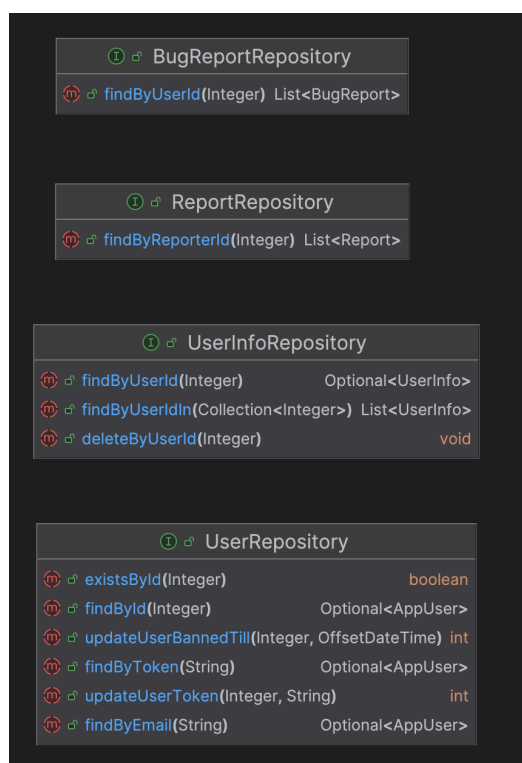


Диаграмма классов services

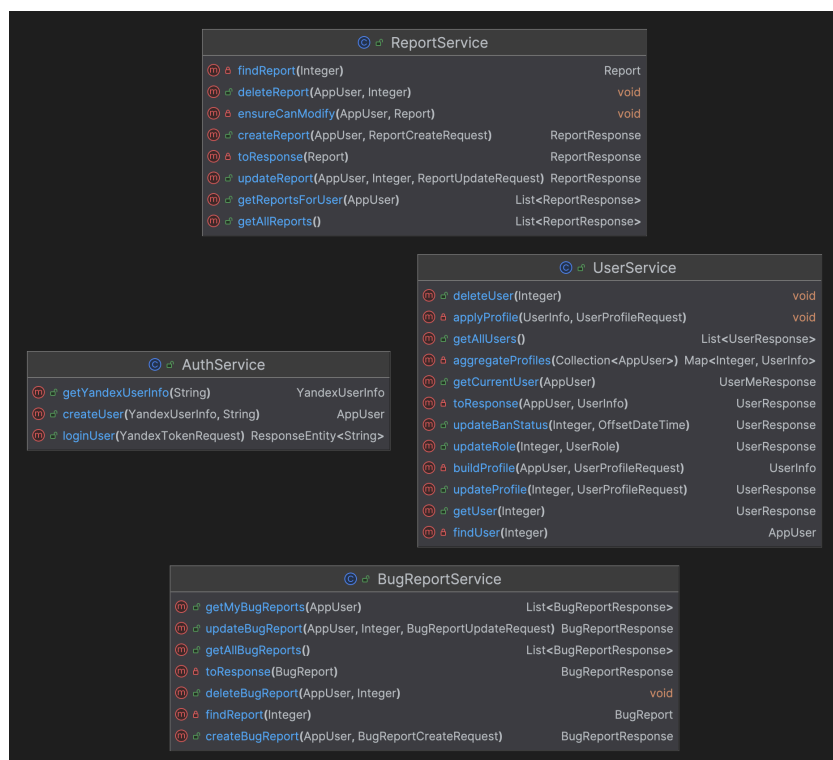


Диаграмма классов Management Service
Общая диаграмма классов



Диаграмма классов config

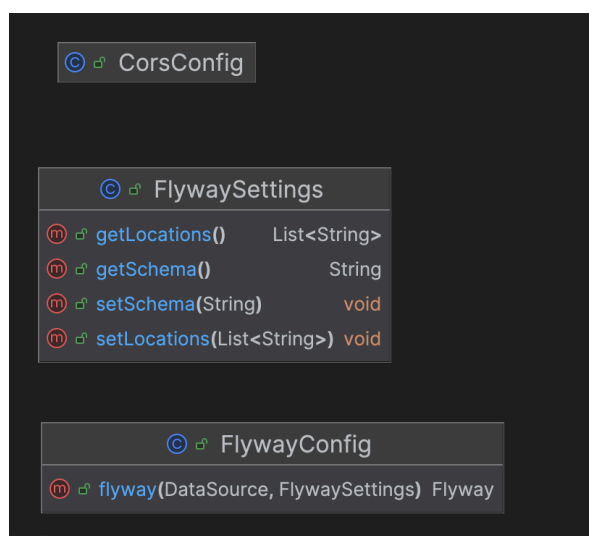


Диаграмма классов controller

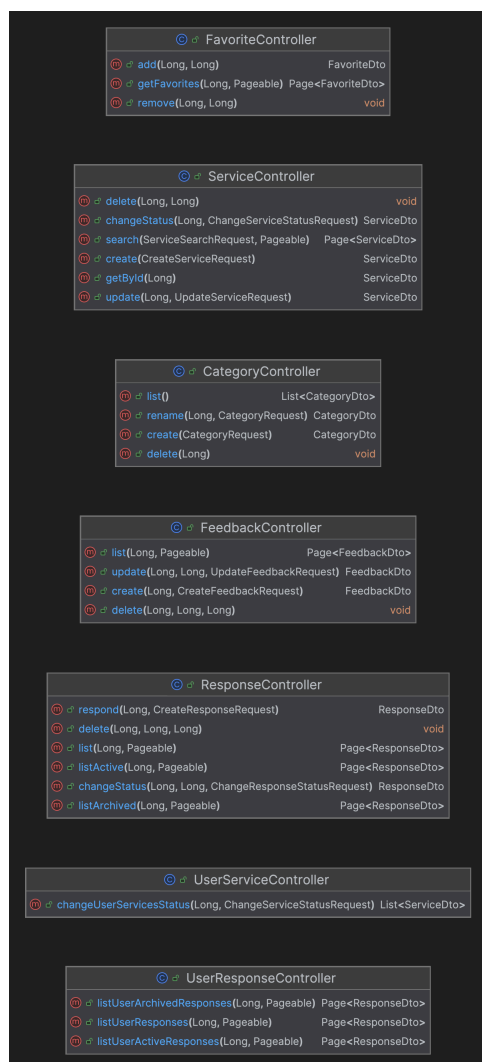


Диаграмма классов models

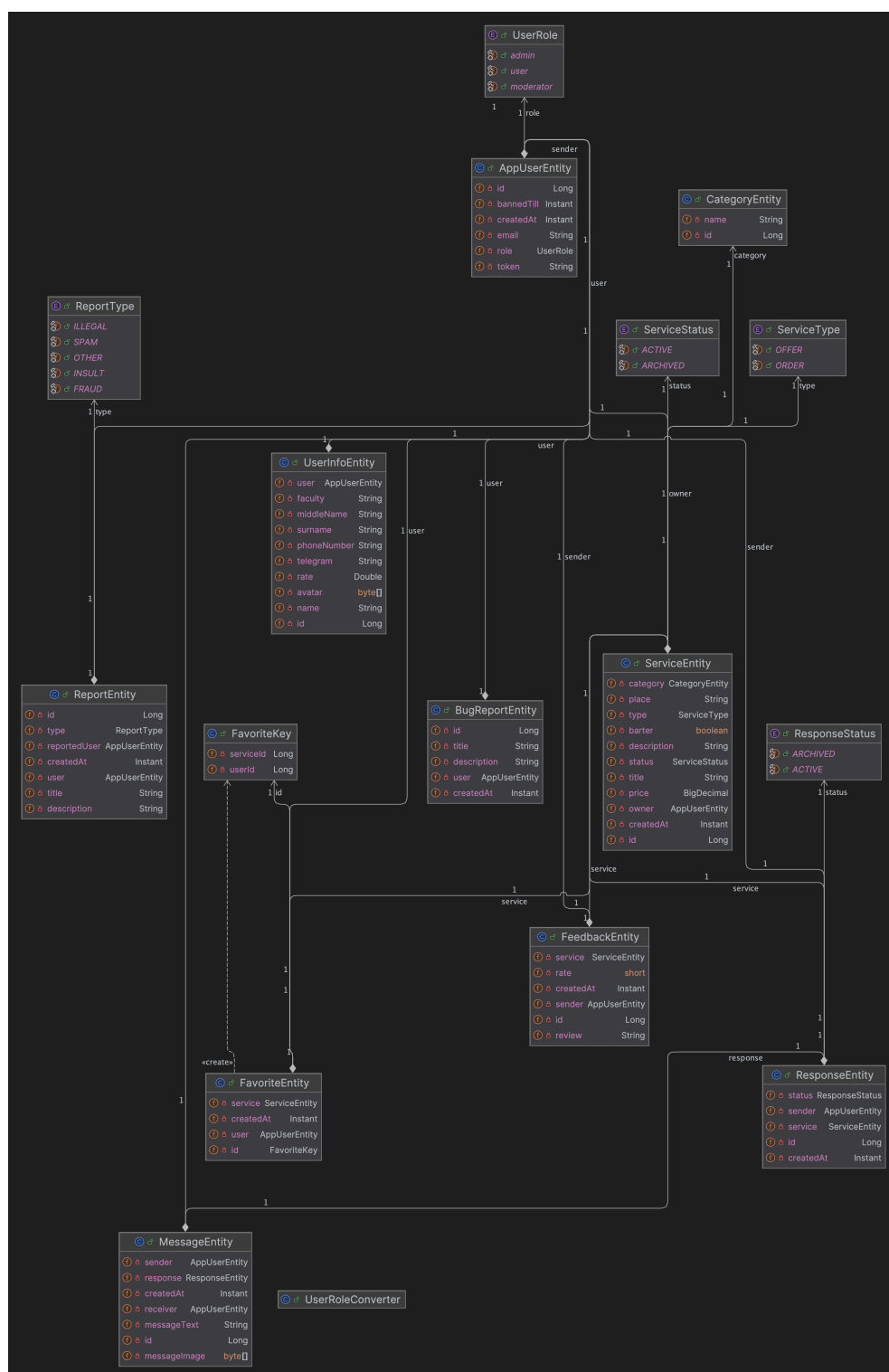


Диаграмма интерфейсов repositories

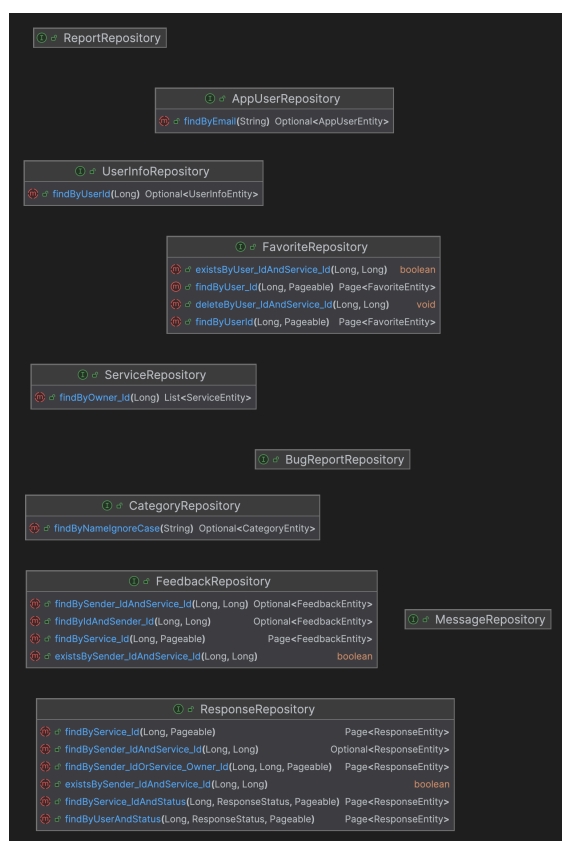


Диаграмма классов services

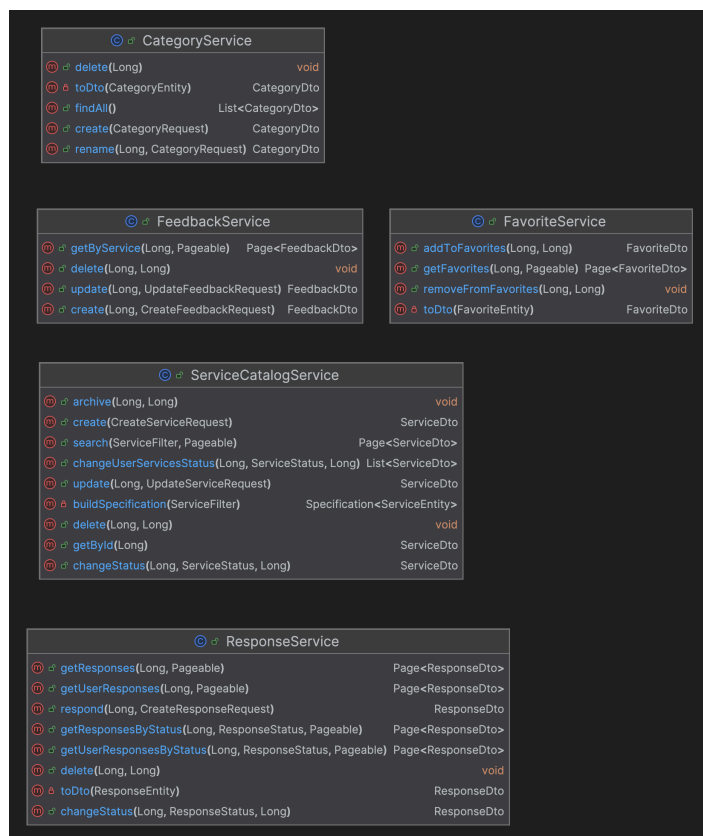


Диаграмма классов Communication Service

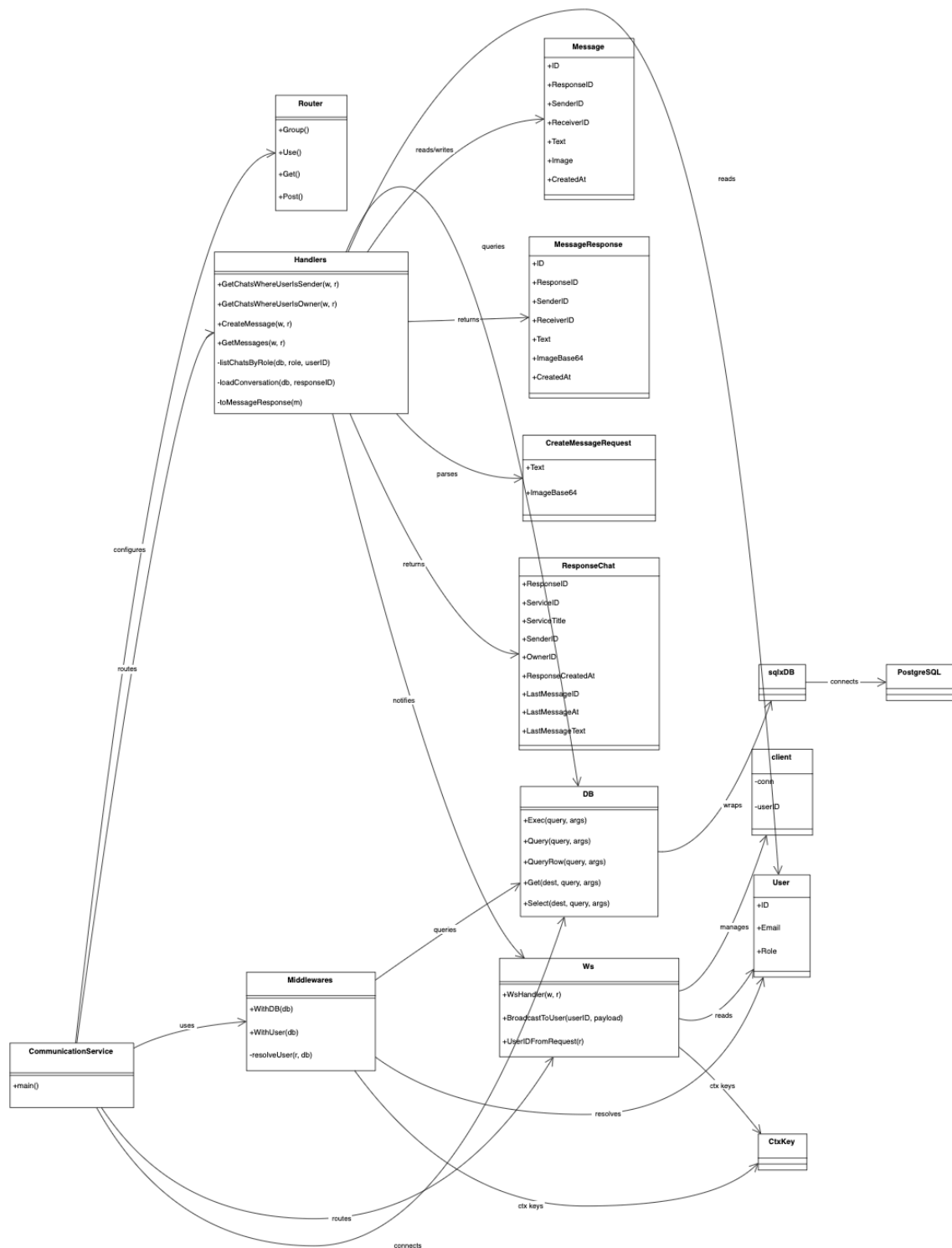
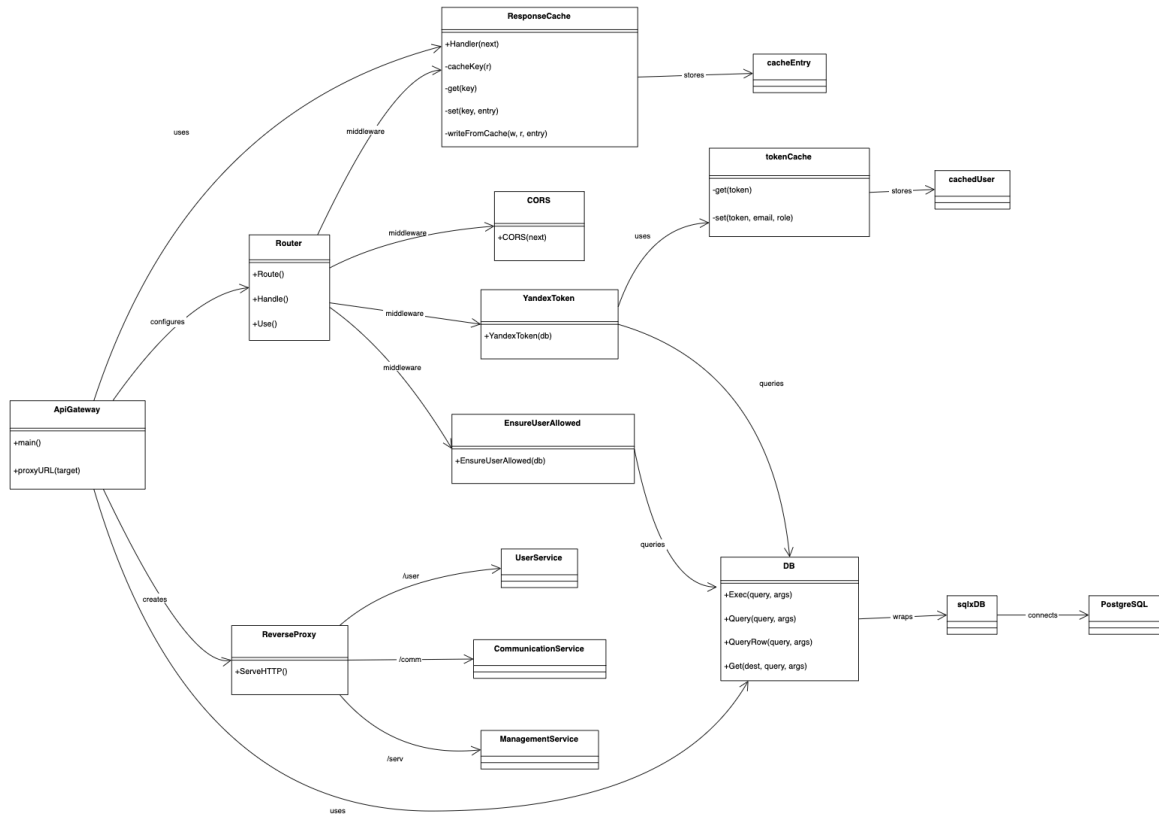


Диаграмма классов API Gateway



Вывод

В ходе выполнения этапа была сформирована и описана архитектура информационной системы HelpMore на уровне взаимодействия компонентов: фронтенд обращается к единой точке входа (API Gateway), которая маршрутизирует запросы в доменные микросервисы (User Management, Management, Communication), а микросервисы выполняют операции с PostgreSQL. На основе разработанной ранее даналогической модели реализован уровень хранения: для ключевых сущностей предметной области построены репозитории, инкапсулирующие доступ к данным. При реализации соблюдено требование использования хранимых функций/процедур pl/pgsql, созданных на втором этапе.

На основании бизнес-процессов из первого этапа и реализованного уровня хранения построен уровень бизнес-логики, который оркестрирует репозитории и реализует основные пользовательские сценарии: публикация и поиск услуг/запросов, управление профилем, отклики и коммуникация в чате, оставление отзывов и формирование репутации, а также обработка обращений для модерации. Таким образом, обеспечена согласованность между требованиями, архитектурой и реализацией, а критические инварианты предметной области централизованы в базе данных, что повышает целостность данных и устойчивость системы к ошибкам реализации на прикладном уровне.

Анализ выполнения требований

№	Требование	Статус	Комментарий
F1.1	Система должна предоставлять неавторизованному пользователю возможность входа по протоколу OAuth 2.0 через Яндекс ID.	Выполнено	
F1.2	Система должна предоставлять неавторизованному пользователю возможность выбора языка в хэпере на всех страницах (включая неавторизованных пользователей).	Выполнено	
F2.1	Система должна предоставлять авторизованному пользователю возможность просмотра открытых предложений и заказов.	Выполнено	
F2.2	Система должна предоставлять авторизованному пользователю возможность просмотра профилей пользователей (ФИО, bio, контакты, рейтинг и отзывы, количество выполненных заказов).	Выполнено	
F2.3	Система должна предоставлять авторизованному пользователю возможность редактирования личного профиля (bio, контакты).	Выполнено (расширено)	Также реализована возможность редактирования ФИО, факультета, аватара профиля
F2.4	Система должна предоставлять авторизованному пользователю возможность создания и публикации заказов (название, описание, категория, стоимость/бартер).	Выполнено	
F2.5	Система должна предоставлять авторизованному пользователю возможность создания публикации предложений услуг (название, описание, категория, стоимость).	Выполнено	
F2.6	Система должна предоставлять авторизованному пользователю возможность поиска и фильтрации открытых заказов (категория, стоимость).	Выполнено	Доказательство: фильтры + запрос с параметрами + корректный список.

F2.7	Система должна предоставлять авторизованному пользователю возможность поиска и фильтрации открытых предложений (категория, рейтинг, стоимость).	Выполнено	
F2.8	Система должна предоставлять авторизованному пользователю возможность отклика на заказы и связи с заказчиками.	Выполнено	
F2.9	Система должна предоставлять авторизованному пользователю возможность отклика на предложения и связи с поставителями.	Выполнено	
F2.10	Система должна предоставлять авторизованному пользователю возможность ведения чата с другими пользователями (обсуждение условий, обмен изображениями).	Выполнено	
F2.11	Система должна предоставлять авторизованному пользователю возможность оставления оценок и текстовых отзывов после выполнения услуги.	Выполнено	
F2.12	Система должна предоставлять авторизованному пользователю возможность просмотра рейтинга и отзывов всех пользователей.	Выполнено частично	Требование не совсем корректно. В системе реализован сценарий что пользователь может просматривать профили только тех пользователей создавших какую либо услугу. Также контакты пользователей скрыты до начала сделки
F2.13	Система должна предоставлять авторизованному пользователю возможность просмотра	Выполнено	

	личной истории заказов и оказанных предложений.		
F2.14	Система должна предоставлять авторизованному пользователю возможность отметки интересных заказов/предложений (избранное).	Выполнено	
F2.15	Система должна предоставлять авторизованному пользователю возможность подачи жалоб на некорректное поведение или нарушение правил.	Выполнено	
F2.16	Система должна предоставлять авторизованному пользователю возможность выхода из аккаунта.	Выполнено	
F2.17	Система должна предоставлять авторизованному пользователю возможность наследовать права неавторизованного пользователя.	Невыполнено	Требование не корректно
F2.18	Система должна предоставлять авторизованному пользователю возможность выбора языка интерфейса в хэдере.	Выполнено	
F3.1	Система должна предоставлять модератору возможность удаления некорректных заказов/предложений.	Выполнено	
F3.2	Система должна предоставлять модератору возможность рассмотрения жалоб пользователей.	Выполнено	
F3.3	Система должна предоставлять модератору возможность выдачи временной блокировки пользователей при нарушении правил.	Выполнено	
F3.4	Система должна предоставлять модератору возможность наследовать права пользователя.	Выполнено	
F4.1	Система должна предоставлять администратору возможность управления категориями услуг (создание, редактирование, удаление).	Выполнено	
F4.2	Система должна предоставлять администратору возможность управления ролями поль-	Выполнено	

	зователей (назначение модераторов, снятие прав).		
F4.3	Система должна предоставлять администратору возможность удаления или восстановления учетных записей пользователей.	Не выполнено	Мягкое удаление не предусмотрено схемой БД, вместо этого была реализована возможность заблокировать пользователя на 1000 лет
F4.4	Система должна предоставлять администратору возможность наследовать права модератора.	Выполнено	

№	Требование	Статус	Комментарий
U1.1	Система должна поддерживать доступность через ARIA-атрибуты.	Выполнено	
U1.2	Система должна обеспечивать мультиязычность интерфейса.	Выполнено	
U1.3	Система должна отображать индикатор процесса при загрузке файлов или больших данных.	Выполнено	
R2.1	Система должна обеспечивать доступность не менее 95% времени в году.	Не выполнено	Проверить невозможно
R2.2	Система должна обеспечивать единый учёт отзывов, оценок, количества выполненных заказов.	Выполнено	
R2.3	Система должна поддерживать мониторинг (с использованием Grafana).	Не выполнено	
P3.1	Система должна предоставлять первые результаты поиска ≤ 2 секунды (с использованием Hibernate search).	Не выполнено	
P3.2	Система должна обеспечивать пропускную способность: не менее 100 запросов просмотров услуг в секунду в течение минуты, не	Выполнено частично	Система выдерживает нагрузку 50 запросов просмотров

	менее 50 публикаций в секунду в течение минуты.		услуг в секунду в течение минуты, 50 публикаций в секунду в течение минуты. Результаты тестирования мы признаём удовлетворительными.
P3.3	Система должна загружать любую страницу за 1–2 секунды при 10 Мбит/с.	Выполнено	
P3.4	Система должна реализовывать кэширование данных.	Выполнено	
S4.1	Система должна быть реализована на основе сервисно-ориентированной архитектуры (SOA).	Выполнено	
S4.2	Система должна разрабатываться по принципу API-first и предоставлять публичные и внутренние API.	Выполнено	
S4.3	Система должна реализовывать RESTful API (JSON) и поддерживать стандартные HTTP-методы (GET, POST, PUT, DELETE).	Выполнено	
S4.4	Система должна обладать документацией к внутреннему и внешнему API (с использованием OpenAPI).	Выполнено	
S4.5	Система должна легко разворачиваться за счёт контейнеризации (с использованием Docker).	Выполнено	
S4.6	Система должна осуществлять мониторинг и алертинг (с использованием Grafana).	Не выполнено	
S4.7	Система должна поддерживать интернационализацию пользовательского интерфейса с возможностью выбора одного из трёх языков: русского, английского и китайского.	Выполнено	
T5.1	Система должна иметь модульные тесты для ключевой бизнес логики.	Выполнено	

T5.2	Система должна иметь настроенный CI/CD через GitHub Actions при коммите в ветку разработки.	Не выполнено	
T5.3	Система должна проверять доступность ключевых сервисов для корректной работы приложения (настройка через docker-compose).	Выполнено	

Процент выполненных требований

Функциональные требования

Строго: 89.29%

Полностью или частично: 91.07%

Не функциональные требования

Строго: 70.00%

Полностью или частично: 72.50%

Общий анализ

Строго: 81.25

Полностью или частично: 83.33%

Вывод

В ходе выполнения курсовой работы была разработана информационная система **HelpMore** — платформа для поиска и предоставления услуг в студенческой среде университета ИТМО. Система обеспечивает полный цикл взаимодействия пользователей: авторизацию, публикацию заказов и предложений, поиск и фильтрацию, отклики, коммуникацию в чате, а также формирование репутации на основе отзывов и оценок. Дополнительно реализованы механизмы модерации и администрирования: обработка жалоб, удаление некорректного контента, временная блокировка пользователей и управление категориями и ролями.

На первом этапе были определены предметная область и основные бизнес-процессы, сформулированы функциональные и нефункциональные требования и построены модели ключевых прецедентов. Это позволило зафиксировать ожидаемое поведение системы и критерии оценки результата разработки.

На втором этапе была спроектирована база данных на основе ER-модели (более 10 сущностей, включая отношение «многие-ко-многим») и реализована даталогическая модель в PostgreSQL. Для обеспечения целостности данных использованы ограничения DDL и триггеры, а критически важные операции вынесены в PL/pgSQL-функции и процедуры. Также были предложены

индексы, обоснованные сценариями использования системы, что повысило эффективность ключевых запросов (лента услуг, фильтрация, переписка, отзывы).

На третьем этапе была сформирована архитектура системы и реализованы уровни хранения и бизнес-логики. Доступ к данным организован через слой репозитория с обязательным использованием разработанных PL/pgSQL-функций/процедур, а слой сервисов реализует основные сценарии работы пользователей и правила предметной области.

На четвертом этапе разработан уровень представления, обеспечивающий выполнение бизнес-процессов, описанных в прецедентах. Итоговый отчет объединяет результаты всех этапов и включает анализ выполнения требований (в том числе ревью по модели FURPS+).

В результате выполнения проекта были закреплены практические навыки проектирования информационных систем, разработки архитектуры, проектирования и оптимизации реляционной базы данных, реализации бизнес-логики и построения пользовательского интерфейса, а также навыки валидации требований на основе фактических результатов реализации.